

蓝牙智能手表综合实验

万旷 3124009598 微电子 2 班

2026/7/13

目录

1 实验目标与任务要求

1. 以 STM32F103C8T6 单片机为核心，搭建一套蓝牙智能手表硬件原型。
2. 移植 FreeRTOS 实时操作系统完成多任务调度，划分独立任务分别处理计时、传感器采集、OLED 屏幕刷新、菜单交互、蓝牙数据收发等功能。
3. 通过 I2C 通信总线驱动 SSD1306 OLED 显示屏与 MPU6050 六轴姿态传感器。
4. 设计五级交互界面，支持旋钮切换表盘、心率、计步、天气、设备信息五个显示页面。
5. 搭配 HC-05 蓝牙模块实现单片机与手机的数据互通，手机端可下发指令读取手表时间、步数、模拟心率数据。
6. 验收标准：硬件接线无错误、OLED 屏幕正常显示、MPU6050 可稳定读取三轴数据、蓝牙模块正常收发指令、计步功能识别准确无大量误计数。

2 实验环境

2.1 硬件设备

- 主控单片机：STM32F103C8T6，内核 ARM Cortex-M3，系统主频 72MHz
- 显示模块：0.96 英寸 SSD1306 OLED 屏幕，分辨率 128×64，I2C 通信接口
- 姿态传感器：MPU6050 六轴模块，集成三轴加速度计与三轴陀螺仪，I2C 接口
- 蓝牙通信模块：HC-05，串口 UART 通信，默认波特率 9600
- 人机交互部件：EC11 旋转编码器 + 独立按键
- 调试工具：ST-Link V2 下载调试器
- 供电系统：3.7V 锂电池 + Type-C 充电升压一体模块
- 辅助器材：面包板、杜邦连接线若干

2.2 软件工具

- STM32CubeMX 6.17: 单片机引脚、外设、操作系统可视化配置, 自动生成底层初始化代码
- VS Code: 工程代码编辑、文本查看
- 编译工具链: ARM GCC 交叉编译器 (gcc-arm-none-eabi) + CMake + Ninja
- 实时操作系统: FreeRTOS, 使用 CMSIS-RTOS v1 标准封装接口
- 手机调试软件: Serial Bluetooth Terminal 蓝牙串口调试工具

3 实验原理与整体方案

3.1 硬件引脚分配与总线连接

I2C1 总线引脚 PB6 (SCL 时钟)、PB7 (SDA 数据) 挂载两块 I2C 设备: SSD1306 OLED 屏幕设备地址 0x3C, MPU6050 传感器设备地址 0x68, 二者地址互不冲突。USART2 串口引脚 PA2 (TX 发送)、PA3 (RX 接收) 对接 HC-05 蓝牙模块。TIM2 定时器配置 1Hz 定时中断, 作为软件实时时钟的计时基准。TIM3 定时器开启编码器模式, 引脚 PA6、PA7 接入 EC11 旋转编码器, 识别旋钮旋转方向与转动步数。HC-05 蓝牙模块 STATE 状态引脚接 PB0, 用于检测蓝牙是否与手机完成配对连接。

3.2 FreeRTOS 多任务划分与优先级设计

工程内一共创建 6 个独立 FreeRTOS 任务, 按任务优先级从高至低排序如下:

- vTask_TimeKeep (高优先级, 栈空间 256 字节) ——每秒发送一次时间更新消息, 同步系统时间
- vTask_Sensor (高于普通优先级, 栈空间 512 字节) ——每 200ms 读取一次 MPU6050 原始数据, 运行计步算法
- StartDisplayTask (普通优先级, 栈空间 1024 字节) ——每 500ms 刷新一次 OLED 屏幕画面
- vTask_Menu (普通优先级, 栈空间 512 字节) ——实时检测编码器旋转与按键按压, 处理页面切换逻辑
- vTask_Bluetooth (低于普通优先级, 栈空间 512 字节) ——每 500ms 轮询串口缓冲区, 解析手机下发蓝牙指令
- StartDefaultTask (普通优先级, 栈空间 128 字节) ——系统空闲后台任务

3.3 任务间数据交互方式

依靠 FreeRTOS 提供的消息队列、互斥锁完成多任务安全数据交互：

- `queueTimeUpdateHandle` ——计时任务向屏幕刷新任务传递当前时分秒、日期数据
- `queueSensorDataHandle` ——传感器任务向屏幕任务推送三轴加速度、步数数据
- `queueMenuMsgHandle` ——菜单交互任务下发页面切换控制指令
- `queueBtCmdHandle` ——蓝牙任务解析手机查询指令，分发对应数据回复
- `oled_mutex_handle` ——I2C 总线互斥锁，避免传感器、屏幕两个任务同时占用总线引发通信错乱

3.4 计步检测算法原理

计步功能依靠加速度传感器输出三轴数据的幅值变化判断人体行走动作。设备静止时，三轴加速度合成幅值稳定在 1g，对应数值 16384 LSB。幅值平方计算公式：

$$\text{mag_sq} = a_x^2 + a_y^2 + a_z^2$$

人行走摆动手腕时，加速度数值会大幅波动，幅值与静止状态差值超过设定阈值即可判定为一步。经过实际调试，阈值设定为 40×1024^2 ，对应 0.4g 加速度变化；同时增加 0.8 秒防抖间隔，避免短时间内重复计数。

4 STM32CubeMX 外设配置步骤

1. 芯片选型：选择 STM32F103C8T6 单片机。
2. RCC 时钟配置：外部高速晶振 HSE=8MHz，PLL9 倍频输出 72MHz 系统时钟；APB1 分频后 36MHz，APB2 分频后 72MHz。
3. 调试与系统配置：调试接口选用串行单线 SWD；系统时钟基准源更换为 TIM1。
4. I2C1 配置：标准 100kHz 通信速率，PB6/PB7 复用为开漏输出，开启引脚内部上拉电阻。
5. I2C2 配置：标准模式 10kHz，引脚 PB10/PB11，预留后期功能拓展使用。
6. USART2 串口：异步通信，波特率 9600，数据位 8、无校验、停止位 1；PA2/PA3 引脚，开启 TX 发送 DMA 通道。
7. TIM2 定时器：内部时钟驱动，预分频 7199，自动重装载值 9999，输出 1Hz 定时中断。

8. TIM3 定时器：双通道编码器模式 (T1+T2)，引脚 PA6、PA7 接入旋转编码器。
9. FreeRTOS 配置：选用 CMSIS-RTOS v1 接口，内存分配策略选择 heap_4。
10. 工程生成设置：输出 CMake 构建工程，也可切换 Makefile 工具链。

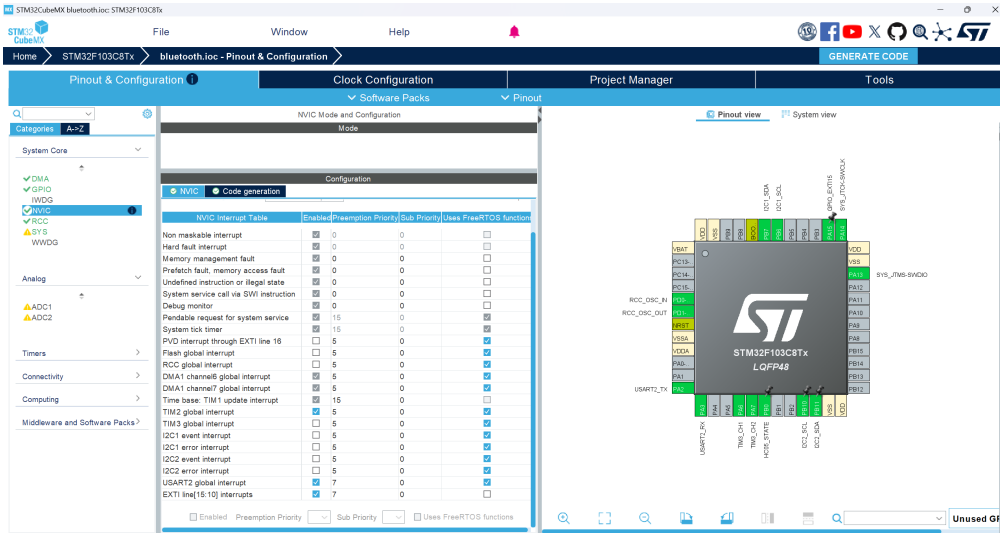


图 1: CubeMX 外设参数配置界面截图

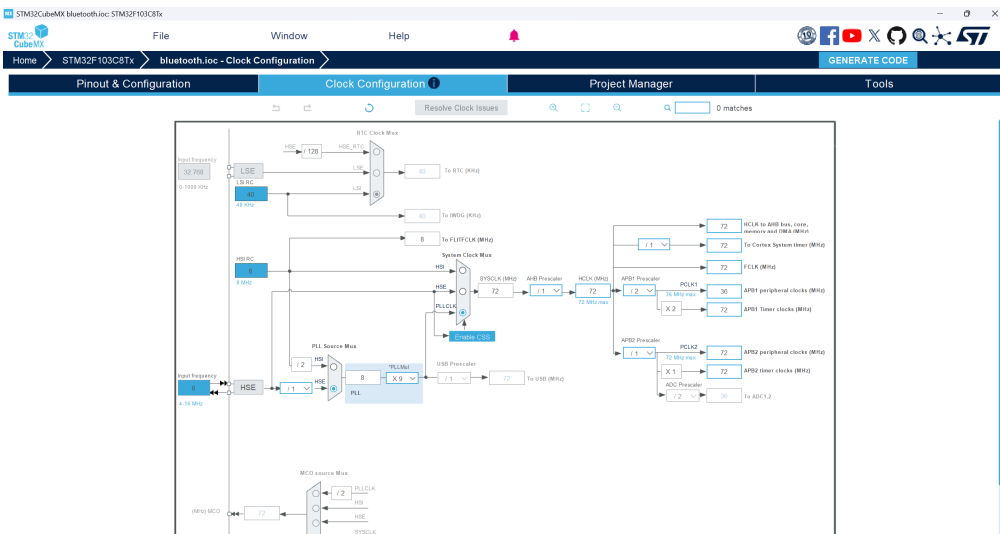


图 2: CubeMX 系统时钟配置界面截图

5 程序设计与核心代码解析

5.1 传感器采集任务主循环

Listing 1: 传感器采集任务主循环代码

```
1 void vTask_Sensor(void const * argument)
2 {
3     MPU6050_Data_t mpu_data;
4     uint8_t sensor_ok = 0;
5     // 初始化传感器并校验硬件连接
6     if (MPU6050_Init(&hi2c1) == HAL_OK) {
7         if (MPU6050_TestConnection(&hi2c1)) sensor_ok = 1;
8     }
9     for(;;) {
10         if (sensor_ok) {
11             // 获取I2C总线互斥锁，独占总线资源
12             if (osMutexWait(oled_mutex_handle, 100) == osOK) {
13                 if (MPU6050_ReadAll(&hi2c1, &mpu_data) == HAL_OK) {
14                     // 将三轴加速度送入计步处理函数
15                     StepCounter_Process(&g_step_counter,
16                                         mpu_data.accel_x, mpu_data.accel_y, mpu_data.
17                                         accel_z);
18                 }
19                 osMutexRelease(oled_mutex_handle);
20             }
21             osDelay(200);
22         }
23     }
```

5.2 计步器核心处理函数

Listing 2: 计步算法：无效数据过滤与幅值阈值判断

```
1 void StepCounter_Process(StepCounter_t *sc,
2     int16_t accel_x, int16_t accel_y, int16_t accel_z)
3 {
4     // 过滤I2C读取失败时输出的全零无效数据
5     if (accel_x == 0 && accel_y == 0 && accel_z == 0) return;
6
7     // 计算三轴加速度幅值平方
8     int32_t mag_sq = (int32_t)accel_x * accel_x +
9                     (int32_t)accel_y * accel_y +
10                    (int32_t)accel_z * accel_z;
11     // 计算当前幅值与静止标准幅值的差值
12     int32_t diff = (mag_sq > 268435456L) ?
```

```
13         (mag_sq - 268435456L) : (268435456L - mag_sq);
14
15     // 幅值超出阈值且防抖计时归零，判定为有效一步
16     if (diff > 40L * 1024L * 1024L && sc->debounce_counter == 0) {
17         sc->total_steps++;
18         sc->daily_steps++;
19         sc->debounce_counter = 4;    // 防抖计数，间隔0.8s内不重复计步
20     }
21     // 防抖计时器自减
22     if (sc->debounce_counter > 0) sc->debounce_counter--;
23 }
```

6 系统整体程序流程图

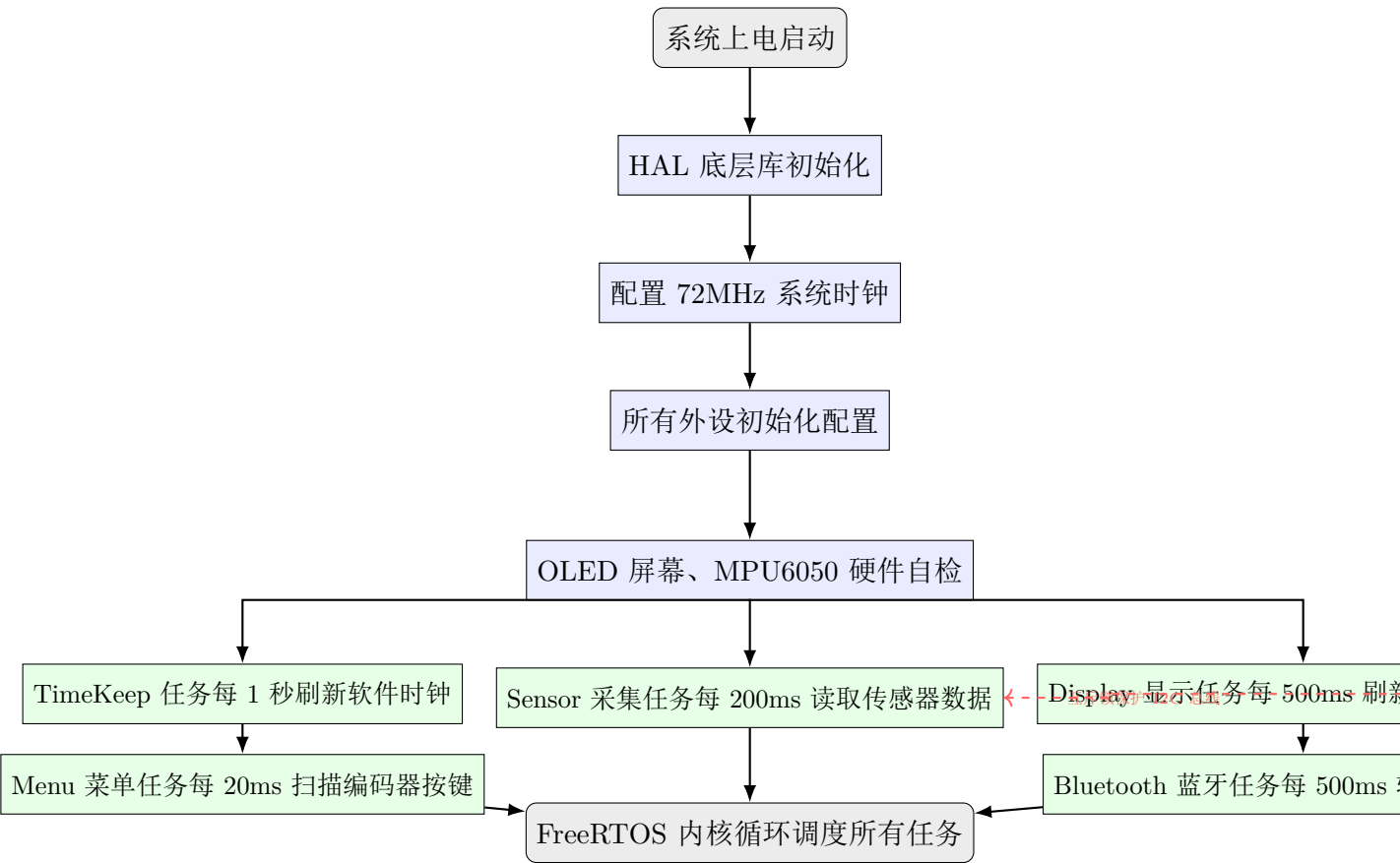


图 3: 蓝牙智能手表完整系统运行流程图

7 实验测试结果与现象说明

- 1. **OLED 屏幕显示效果：**设备开机后先打印“SmartWatch”开机画面，随后自动跳转至主表盘界面，界面展示大号时间、日期基础信息。旋转编码器旋钮可循环切换五个

- 功能页面：表盘、心率、计步、天气、设备信息；屏幕顶部状态栏固定显示蓝牙连接状态、剩余电量、当前时间，底部设置页面点位指示器。
2. **MPU6050 传感器测试：**将手表固定静置时，设备信息页面三轴加速度数值保持稳定；人为摆动手腕，步数数据正常累加；静置放置后步数不再上涨，无自动误加计数。
 3. **蓝牙通信测试：**手机蓝牙配对 HC-05 模块，配对密码 1234。手机发送指令 R，手表返回当前时间字符串 TIME:10:30:45；发送 S 返回总步数 STEP:xxx；发送 H 返回模拟心率 HR:xxx，所有指令收发逻辑符合设计预期。
 4. **计步功能实测：**正常摆臂行走 10 步左右，屏幕步数同步增加；阈值 40×1024^2 可有效过滤桌面轻微震动带来的干扰，同时不会漏掉正常行走动作。

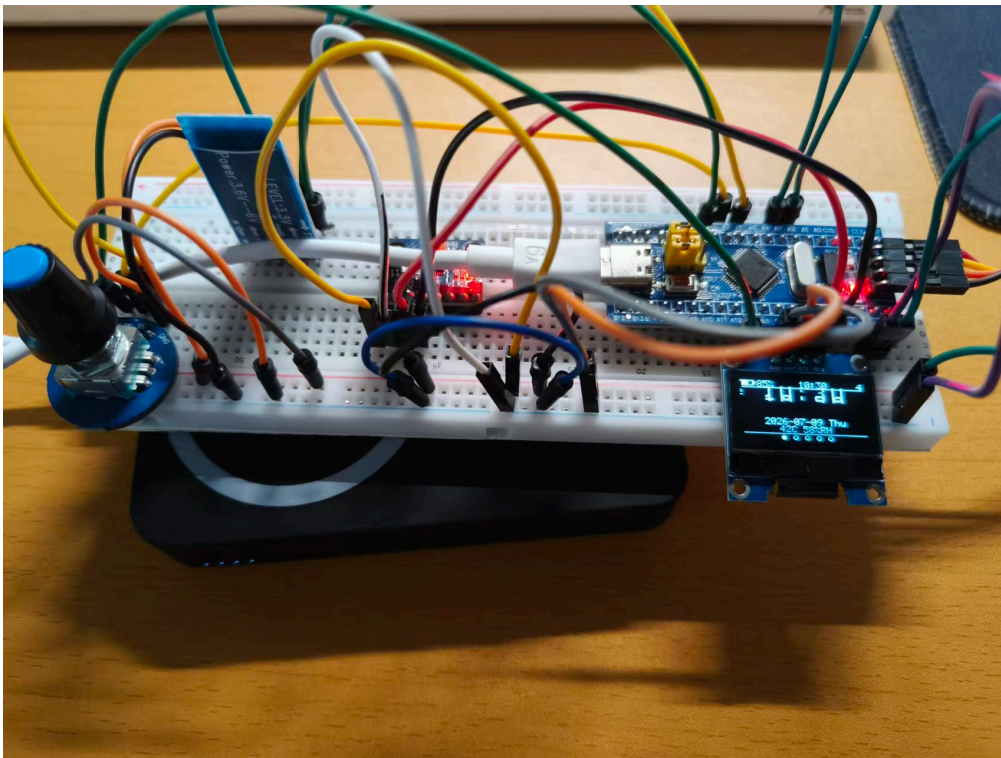


图 4: 智能手表硬件实物整体照片

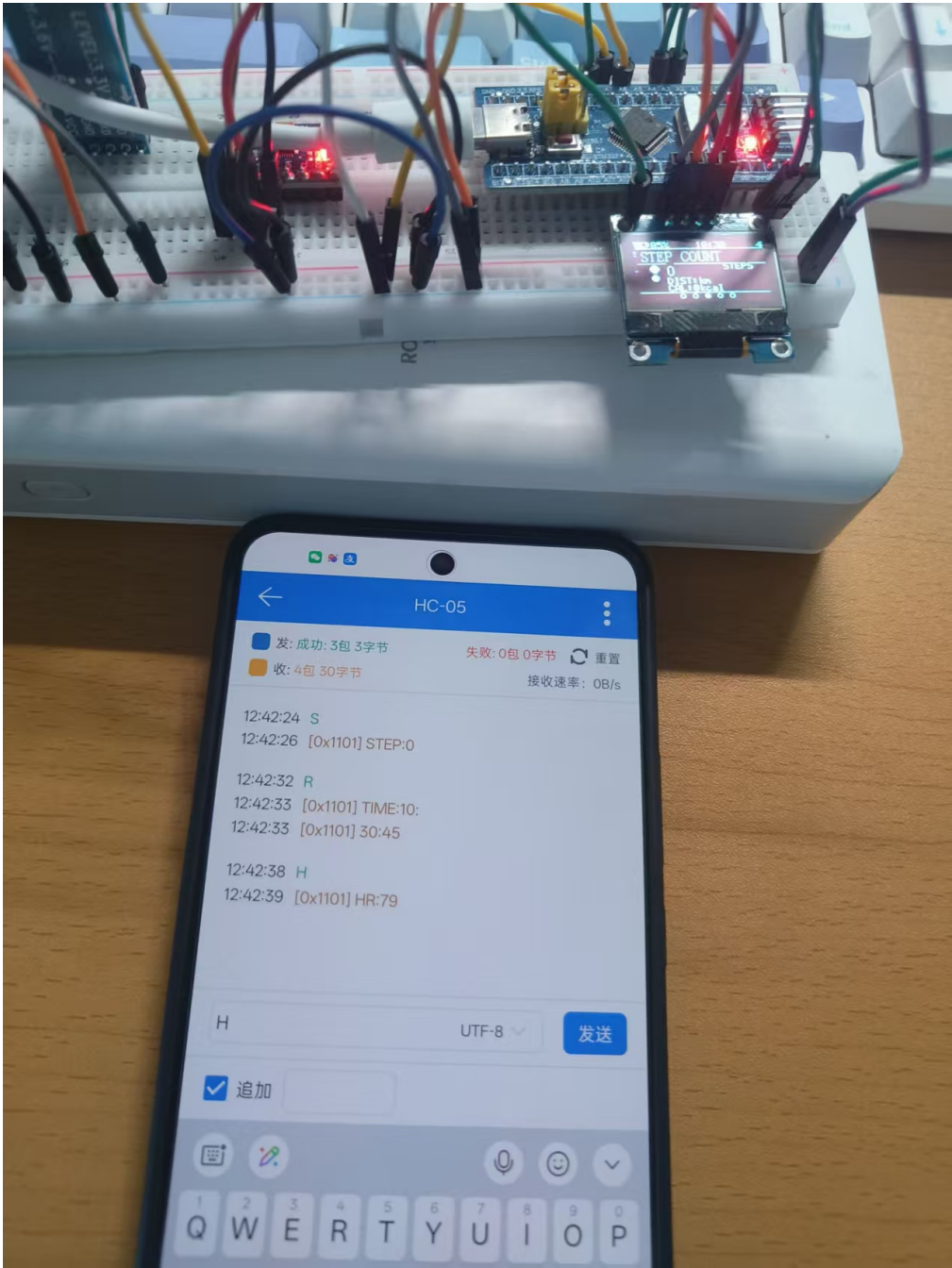


图 5: 手机蓝牙调试软件通信测试截图

8 故障现象与调试解决记录

故障现象	排查与修复过程
MPU6050 读取失败	OLED 屏幕提示传感器读取失败，读取 WHO_AM_I 寄存器返回 0x74，并非传感器标准地址 0x68。故障原因：1.I2C1 总线引脚未开启内部上拉电阻；2. 传感器校验函数仅判断通信状态，未校验设备 ID。修复方案：开启 GPIO 内部上拉，增加 WHO_AM_I 寄存器校验逻辑。
手表静置步数自动上涨	初始阈值设置为 10×1024^2 ，微小震动就会触发计步，同时 I2C 读取失败输出全零数据也会误计数。修复方案：函数入口过滤三轴全零无效数据；提高加速度阈值至 40×1024^2 ；新增 0.8 秒防抖计时。
蓝牙模块无法收发数据	HC-05 出厂默认波特率 9600，最初代码串口配置 38400；同时串口 DMA 接收与轮询读取逻辑冲突。修复方案：串口波特率修改为 9600，关闭串口 DMA 接收通道；舍弃 STATE 引脚检测逻辑，使用阻塞式 HAL_UART_Transmit 发送数据。

表 1: 实验故障排查记录表

9 课后思考题解答

1. OLED 屏幕与 MPU6050 传感器为什么可以挂载在同一条 I2C 总线上？I2C 总线属于多设备共享总线架构，依靠 7 位设备地址区分总线上不同外设。本实验中 OLED 屏幕地址 0x3C，MPU6050 传感器地址 0x68，两个设备地址完全独立，不存在冲突。主机发送地址字节后，只有地址匹配的从设备会应答并响应后续读写指令。
2. FreeRTOS 多任务共用一条 I2C 总线，如何避免多任务同时访问引发通信错乱？使用 FreeRTOS 互斥锁 Mutex 对 I2C 总线做资源保护。传感器任务、屏幕刷新任务在读写 I2C 总线前，先调用 osMutexWait 获取锁；总线操作完成后执行 osMutexRelease 释放锁。同一时间仅一个任务能持有互斥锁访问总线，其余任务进入阻塞等待；互斥锁自带优先级继承机制，还能规避高优先级任务被低优先级任务阻塞的优先级翻转问题。
3. 计步算法通过哪些手段避免误计数？算法设置三层过滤机制减少误触发：过滤 I2C 读取故障产生的三轴全零无效数据；加速度合成幅值波动必须超过设定阈值，屏蔽桌面微小震动噪声；增加防抖计时器，两次有效计步间隔至少 0.8 秒，过滤连续小幅晃动。

10 实验总结

- 本次实验以 STM32F103C8T6 单片机搭配 FreeRTOS 实时操作系统，完成蓝牙智能手表全套功能原型开发。借助 STM32CubeMX 完成所有外设可视化配置，基于 CMSIS-RTOS 接口划分 6 个独立任务，分别实现计时、传感器采集、屏幕显示、旋钮菜单交

互、蓝牙数据交互功能。调试阶段依次解决 I2C 上拉缺失导致传感器读取出错、计步阈值过低误计数、串口 DMA 冲突蓝牙无响应等故障，最终整套系统稳定运行，全部达到实验设计要求。后续还可以拓展功能：固件 OTA 无线升级、SD 卡本地存储运动数据、卡尔曼滤波优化姿态解算等。

- 硬件物料总成本约 80 元，实验全程无元器件烧毁、硬件损坏情况。
- 个人工作内容：独立完成硬件接线、OLED 底层驱动、MPU6050 数据读取、蓝牙收发逻辑、计步算法代码编写，以及全部软硬件故障调试工作。
- 关键故障总结：单独将 MPU6050 挂载 I2C 总线时无法读取数据，排查后确认是总线缺少上拉电阻；OLED 模块自带硬件上拉，将传感器与屏幕共用一条 I2C 总线后通信恢复正常。
- 实验成果：完整实现智能手表基础功能，包含时间实时显示、运动步数统计、模拟心率展示、简易天气页面、设备参数查看；支持手机蓝牙下发指令查询手表数据，双向数据通信功能调试完成。
- 现有不足：当前仅支持手机单向读取手表本地数据，手表无法接收手机下发的实时消息同步至屏幕显示；后续可完善蓝牙接收解析逻辑，实现手机与手表双向数据传输。

11 附录工程配套资料

- STM32 配置工程文件：bluetooth.ioc
- 实时操作系统版本：FreeRTOS 10.x，采用 CMSIS-RTOS v1 封装层
- 编译工具链：arm-none-eabi-gcc 交叉编译器 + CMake + Ninja 构建工具
- 参考文档：STM32F1xx HAL 库手册、MPU6050 寄存器数据手册、SSD1306 屏幕规格书、HC-05 蓝牙 AT 指令手册